

PYTHON PROFISSIONAL

INFORMAÇÕES PARA O MUNDO



Caroline Perez Gonçalves
2026

PYTHON PROFISSIONAL

Caroline Perez Gonçalves

SUMÁRIO

APRESENTAÇÃO.....	1
INTRODUÇÃO	2
AMBIENTES VIRTUAIS (VENV)	3
GERENCIANDO PACOTES COM PIP	5
BOAS PRÁTICAS.....	7
INTRODUÇÃO AO GIT	10
HORA DE PRATICAR!	13
AGRADECIMENTO.....	18
CONTATO.....	19

APRESENTAÇÃO

Aos que não me conhecem, meu nome é Caroline. Sou formada em Análise e Desenvolvimento de Sistemas, pós-graduada em Inteligência Artificial e Machine Learning e possuo MBA em Gerenciamento de Projetos.

Além da minha experiência profissional com tecnologia, projetos e equipes multidisciplinares, também atuo como instrutora. Sempre acreditei que o conhecimento deve ser compartilhado de forma simples, leve e acessível, e foi com esse propósito que esta apostila foi desenvolvida.

O intuito deste material é mostrar como o Python é utilizado no mercado de trabalho e apresentar ferramentas, práticas e conceitos que fazem parte da rotina de desenvolvedores profissionais. Mais do que aprender novos recursos da linguagem, o objetivo agora é entender como organizar projetos, utilizar boas práticas e conhecer tecnologias que tornarão seu desenvolvimento mais seguro, eficiente e preparado para desafios reais.

Ao final desta apostila, espero que você se sinta mais confiante para desenvolver seus próprios projetos, continuar evoluindo na linguagem e dar os próximos passos na sua carreira. Lembre-se de que nenhum desenvolvedor sabe tudo. O que faz a diferença é a vontade de aprender continuamente, praticar e nunca deixar de explorar novas possibilidades.

Vamos aos estudos! Mas, antes, sente-se, relaxe e sinta-se em casa. Quero que este material continue sendo como uma boa conversa entre amigos, tornando o aprendizado mais leve, agradável e, principalmente, significativo.

Boa sorte e muito sucesso!

INTRODUÇÃO

Até aqui, você aprendeu os principais conceitos da linguagem Python. Criou seus primeiros programas, trabalhou com estruturas condicionais, laços de repetição, funções, módulos e desenvolveu aplicações capazes de manipular arquivos e resolver problemas do dia a dia.

Mas existe uma pergunta que costuma surgir nesse momento. Eu sei que essa trilha talvez tenha sido a mais chata por ter tantos questionamentos, mas é exatamente nesse momento que as dúvidas surgem e não tem como fugir. Então, vamos aos esclarecimentos.

"O que muda quando começamos a desenvolver projetos maiores?"

A resposta para essa pergunta tem um segredinho.

Muita gente acredita que programar é apenas escrever código. Mas a verdade é que, conforme os projetos crescem, surgem novas preocupações. Como organizar o ambiente de desenvolvimento? Como instalar bibliotecas sem causar conflitos? Como manter o projeto organizado e facilitar futuras manutenções?

Como assim, Carol?

Imagine que você esteja desenvolvendo vários projetos ao mesmo tempo. Cada um deles pode utilizar bibliotecas diferentes, versões diferentes e até tecnologias diferentes. Sem organização, é muito fácil encontrar problemas que poderiam ser evitados.

É justamente por isso que os desenvolvedores utilizam ferramentas e boas práticas para organizar seus projetos desde o início. Elas tornam o desenvolvimento mais seguro, facilitam o trabalho em equipe e ajudam a manter cada aplicação independente das demais.

Ao longo desta apostila, você conhecerá algumas dessas ferramentas e entenderá por que elas fazem parte da rotina de praticamente todo desenvolvedor Python. Pode parecer um detalhe agora, mas são esses pequenos cuidados que tornam um projeto muito mais profissional.

Para que esta introdução não fique muito extensa, vamos explorar cada assunto no momento certo. E, como todo bom aprendizado, vamos começar pelo começo.

Vamos lá.

AMBIENTES VIRTUAIS (VENV)

Até aqui, nossos programas foram relativamente simples. Criamos arquivos, utilizamos bibliotecas, consumimos APIs e desenvolvemos aplicações cada vez mais completas. Mas existe um detalhe muito importante que ainda não discutimos.

Imagine que você trabalha em duas empresas diferentes. Haja saúde mental.

Na primeira, um projeto utiliza uma versão mais antiga de uma determinada biblioteca. Já o segundo projeto precisa da versão mais recente dessa mesma biblioteca.

E agora?

Se você atualizar a biblioteca, o primeiro projeto pode deixar de funcionar. Mas, se mantiver a versão antiga, talvez o segundo projeto apresente problemas.

Parece uma situação complicada, não é?

Pode acreditar, isso acontece com muita frequência no desenvolvimento de software.

Foi justamente para resolver esse tipo de problema que surgiram os **ambientes virtuais**.

Como assim, Carol?

Imagine que você tenha um armário com várias gavetas.

Em vez de misturar todas as suas roupas em um único lugar, você separa cada categoria em uma gaveta diferente. Assim, fica muito mais fácil encontrar o que precisa e tudo permanece organizado.

Os ambientes virtuais funcionam exatamente dessa forma.

Cada projeto possui seu próprio espaço, com suas próprias bibliotecas e configurações, sem interferir nos demais projetos existentes no computador.

Em outras palavras, um ambiente virtual cria uma área isolada para cada aplicação. Isso significa que você pode instalar bibliotecas diferentes para projetos diferentes, sem correr o risco de gerar conflitos.

Hoje em dia, trabalhar com ambientes virtuais deixou de ser uma boa prática e passou a ser praticamente uma exigência no mercado. Grande parte das empresas utiliza esse recurso justamente para garantir que todos os desenvolvedores trabalhem com as mesmas versões de bibliotecas e dependências.

No Python, o ambiente virtual pode ser criado utilizando o módulo **venv**, que já acompanha a instalação da linguagem. Ou seja, não é necessário instalar nenhuma ferramenta adicional.

Para criar um ambiente virtual, basta abrir o terminal na pasta do projeto e executar o seguinte comando:

```
python -m venv venv
```

E é interessante entendermos o que esse comando quer dizer. Vamos por partes:

- python executa o interpretador da linguagem.
- -m informa que desejamos executar um módulo do Python.
- venv é o módulo responsável por criar ambientes virtuais.
- O último venv é apenas o nome da pasta onde o ambiente será criado. Você pode utilizar outro nome, mas esse é o padrão adotado pela maioria dos desenvolvedores.

Após a execução do comando, uma nova pasta será criada dentro do projeto. Ela conterá todos os arquivos necessários para o funcionamento daquele ambiente virtual.

Mas existe um detalhe importante.

Criar o ambiente não significa que ele já está sendo utilizado.

Antes de instalar bibliotecas ou executar o projeto, precisamos **ativar** esse ambiente.

No Windows, o comando é: `venv\Scripts\activate`

Já no Linux e no macOS, utilizamos: `source venv/bin/activate`

Depois da ativação, normalmente o terminal exibirá o nome do ambiente entre parênteses, indicando que ele está ativo.

`(venv) C:\Projetos\MeuSistema>`

A partir desse momento, qualquer biblioteca instalada ficará disponível apenas para aquele projeto.

Quando terminar o trabalho, também é possível desativar o ambiente virtual, utilizando: `deactivate`

Isso fará com que o terminal volte ao estado normal.

Talvez você esteja pensando:

"Mas eu realmente preciso usar ambientes virtuais em projetos pequenos?"

A resposta é não.

Se você estiver apenas praticando ou criando pequenos exercícios, provavelmente não fará muita diferença.

Mas, quando começar a desenvolver aplicações maiores ou trabalhar em equipe, utilizar ambientes virtuais será praticamente obrigatório. Eles tornam os projetos mais organizados, evitam conflitos entre bibliotecas e facilitam muito a colaboração entre desenvolvedores.

E é importantíssimo entendermos que os detalhes fazem diferença. Se as coisas existem, tem um porquê. Utilizem!

GERENCIANDO PACOTES COM PIP

Vimos como criar um ambiente virtual para isolar nossos projetos. Mas como vamos utilizar uma biblioteca que não acompanha a instalação do Python? Como fazer para adicioná-la ao seu projeto?

É justamente aí que entra o **pip**.

O pip é o gerenciador de pacotes oficial do Python. Em outras palavras, ele é a ferramenta responsável por instalar, atualizar e remover bibliotecas que poderão ser utilizadas nos seus programas.

Lembra quando estudamos o consumo de APIs? Para utilizar a biblioteca requests, foi necessário executarmos um comando no terminal.

```
pip install requests
```

Foi exatamente o pip que realizou todo esse trabalho.

Ele acessou um repositório na internet, fez o download da biblioteca, instalou todos os arquivos necessários e deixou tudo pronto para que pudéssemos utilizá-la no projeto.

Isso parece um processo simples. E, na verdade, é mesmo. Muitos outros processos, dizemos que é simples para causar uma sensação de conforto. Mas, esse, é realmente simples! hahaha

Imagine fazer tudo isso manualmente?

Seria necessário localizar a biblioteca, baixar os arquivos corretos, copiar tudo para as pastas certas e ainda verificar se outras bibliotecas também precisariam ser instaladas.

Felizmente, o pip faz todo esse trabalho automaticamente.

Instalando uma biblioteca

O comando mais utilizado é justamente o de instalação.

```
pip install requests
```

Depois de alguns segundos, a biblioteca estará disponível para utilização.

Agora basta importá-la normalmente no código.

```
import requests
```

Fim!

Atualizando uma biblioteca

As bibliotecas também evoluem. Não pense que elas vão ficar iguais para todo sempre. Novas funcionalidades são adicionadas, erros são corrigidos e melhorias são

implementadas constantemente.

Se você desejar atualizar uma biblioteca instalada, basta utilizar o comando:

```
pip install --upgrade requests
```

O pip verificará se existe uma versão mais recente e fará a atualização automaticamente.

Fim também!

Removendo uma biblioteca

Nem sempre uma biblioteca continuará sendo utilizada. Quando isso acontecer, também podemos removê-la.

```
pip uninstall requests
```

Antes da remoção, o pip solicitará uma confirmação para evitar exclusões acidentais.

Fica chato eu escrever “fim também” novamente?

Descobrimos quais bibliotecas estão instaladas

Imagine que você abriu um projeto criado há alguns meses e não lembra quais bibliotecas estão instaladas.

Não precisa adivinhar.

O próprio pip pode mostrar essa informação. Basta usar o comando: `pip list`

Esse comando exibirá uma lista com todas as bibliotecas disponíveis no ambiente virtual ativo.

É um recurso bastante útil, principalmente quando trabalhamos em projetos maiores.

O arquivo requirements.txt

Agora vou apresentar um recurso muito utilizado por desenvolvedores profissionais. Usei esses dias e me dei conta como é importante e útil.

Imagine que você terminou um projeto e deseja enviá-lo para outro colega da equipe. Como ele saberá quais bibliotecas precisa instalar?

É justamente para isso que existe o arquivo **requirements.txt**.

Ele funciona como uma lista de todas as dependências do projeto. Podemos criá-lo facilmente utilizando o comando: `pip freeze > requirements.txt`

O resultado será algo parecido com isto:

```
requests==2.32.0
```

```
numpy==2.1.0
```

```
pandas==2.3.1
```

Sempre que outra pessoa receber esse projeto, bastará executar:

```
pip install -r requirements.txt
```

O pip instalará automaticamente todas as bibliotecas listadas no arquivo. Percebe como isso facilita o trabalho em equipe? Alias, o seu trabalho também!

Em vez de instalar cada biblioteca manualmente, todo o ambiente pode ser preparado com apenas um comando.

Hoje em dia, praticamente todos os projetos profissionais utilizam um arquivo requirements.txt.

BOAS PRÁTICAS

Sua memória é boa o suficiente para você ter certeza que vai lembrar de um código criado há 6 meses? E há 12 meses? E se alguém precisar trabalhar em cima da sua criação, como vai ser?

Provavelmente vamos esbarrar em algum momento com códigos que não sabemos como foi desenvolvido.

Imagine só você receber um projeto criado por outra pessoa... Você abre o código e encontra variáveis chamadas x, a1, teste, valor2 e funções com centenas de linhas. Além disso, não existe um único comentário explicando o funcionamento do sistema.

Confesso que não seria uma experiência muito agradável. Você ia sentir algumas partes do corpo pinicando de suor.

E pode acreditar: isso acontece com mais frequência do que você imagina.

É justamente por isso que existem as **boas práticas de programação**.

Escrever um programa que funciona é importante. Mas escrever um programa que outra pessoa consiga entender também é.

Na prática, um bom código não é apenas aquele que resolve um problema. Ele também deve ser organizado, fácil de ler e simples de manter.

Pense em um livro.

Se todas as palavras estivessem escritas sem espaços, sem pontuação e sem parágrafos, você conseguiria entender o conteúdo? Ia ser uma maluquice.

Provavelmente conseguiria... mas daria muito mais trabalho.

Com o código acontece exatamente a mesma coisa.

Quanto mais organizado ele estiver, mais fácil será compreender seu funcionamento.

Escolha nomes que façam sentido

Um dos erros mais comuns entre iniciantes é utilizar nomes genéricos para variáveis e funções. Eu já fiz isso com abreviações da minha cabeça e confesso que não lembrei depois. Ainda bem que eram projetos pessoais. Veja estes exemplos:

```
a = 150
```

```
b = 0.10
```

```
c = a * b
```

Esse código funciona. Mas responda com sinceridade... O que raios representa a variável `a`? E a `b`?

Agora observe este outro exemplo:

```
valor_produto = 150
```

```
desconto = 0.10
```

```
valor_final = valor_produto * desconto
```

Agora sim está muito mais fácil de entender. Tão fácil, que não precisa de explicação. Os próprios nomes já mostram o que está acontecendo.

Evite repetir código

Lembra das funções?

Elas não existem apenas para deixar o programa mais bonito. Elas ajudam a evitar repetição.

Imagine copiar o mesmo trecho de código dez vezes.

Se um dia for necessário alterá-lo, você precisará fazer a mesma modificação em todos os lugares. Agora imagine que esse código esteja dentro de uma função.

Bastará alterar uma única vez.

Muito melhor, não acha?

Sempre que perceber que está copiando e colando o mesmo código diversas vezes, pare e pergunte em voz alta:

"Será que isso não poderia virar uma função?"

Na maioria das vezes, a resposta será sim. Sua vida será mais leve!

Escreva comentários quando fizer sentido

Os comentários são excelentes para explicar trechos de código mais complexos. Mas isso não significa comentar todas as linhas. Não se empolgue!

Veja este exemplo:

```
# Soma dois números
```

```
resultado = numero1 + numero2
```

Esse comentário é realmente necessário? EU acredito seriamente que não. O próprio código já deixa claro o que está acontecendo.

Agora imagine uma regra de negócio mais específica.

```
# Clientes com mais de 60 anos possuem desconto de 15%
```

```
if idade >= 60:
```

```
    desconto = 0.15
```

Apesar do código também ser explicativo no que está acontecendo, o comentário ajuda a explicar **o motivo** daquela regra existir.

Uma boa dica é esta: comente **o porquê** e não apenas **o que** o código faz.

Organize seu código

Outra boa prática é manter o código bem organizado.

Procure utilizar espaços em branco para separar blocos de instruções.

Agrupe funções relacionadas.

Evite criar arquivos enormes quando o projeto começar a crescer.

Esses pequenos cuidados deixam a leitura muito mais agradável.

Lembre-se: Você passará muito mais tempo lendo código do que escrevendo código.

E esse trecho aqui foi escrito pensando em te mostrar que, pular linhas, realmente deixa a aparência mais limpa.

Siga um padrão

Imagine um livro em que cada capítulo utiliza uma fonte diferente, tamanhos variados

e estilos completamente distintos. Seria bastante confuso (e feio). Nos projetos acontece algo parecido.

Procure seguir sempre o mesmo padrão de nomenclatura, identificação e organização.

O Python possui um guia oficial chamado **PEP 8**, que reúne diversas recomendações sobre como escrever um código mais limpo e consistente.

Você não precisa decorar todas essas regras agora.

O mais importante é saber que esse guia existe e consultá-lo sempre que tiver dúvidas.

E é super, hiper, hiper e hiper importante seguir a indentação! A indentação são os recuos que separam blocos/ ações, lembra?

INTRODUÇÃO AO GIT

Esse talvez seja o assunto mais polêmico para mim. É aquele “faça o que eu digo, mas não o que eu faço”. E, no decorrer dele, você vai entender o motivo, pois eu faço questão de comentar muito!

Se eu te perguntasse qual foi a última alteração que você fez em um arquivo importante, você conseguiria responder?

Agora imagine outra situação.

Você passou três dias desenvolvendo um projeto. Resolveu fazer algumas alterações e, sem perceber, acabou apagando uma parte importante do código. E agora?

Se você não tiver uma cópia de segurança, provavelmente terá que refazer tudo novamente.

Pode acreditar. TODO desenvolvedor já passou por uma situação parecida.

É justamente para evitar esse tipo de problema que existe o **Git**.

Imagine que você está escrevendo um livro. Todos os dias faz pequenas alterações, corrige alguns trechos, adiciona novos capítulos e remove partes que já não fazem mais sentido.

Não seria interessante conseguir voltar para a versão de ontem caso algo desse errado? Ou comparar duas versões diferentes do mesmo texto?

O Git faz exatamente isso.

Ele registra as alterações realizadas no projeto ao longo do tempo, permitindo acompanhar a evolução do código e recuperar versões anteriores sempre que necessário.

Por isso, o Git é conhecido como um **sistema de controle de versão**.

Hoje em dia, ele é utilizado por empresas de todos os tamanhos e faz parte da rotina da grande maioria dos desenvolvedores. Inclusive, é requisito em muitas vagas, viu?

Por que utilizar o Git?

No começo da sua jornada, talvez seus projetos sejam pequenos e desenvolvidos apenas por você. Nessa situação, pode até parecer que o Git não faz muita diferença.

Mas imagine um sistema desenvolvido por uma equipe com dez programadores.

Todos modificando arquivos ao mesmo tempo. Como controlar essas alterações? Como saber quem fez determinada mudança? Como recuperar uma versão anterior caso algo pare de funcionar?

É justamente aí que o Git se torna indispensável.

Ele registra todo o histórico do projeto e permite que várias pessoas trabalhem no mesmo código de maneira muito mais organizada.

E vale reforçar: é indispensável.

Instalando o Git

O primeiro passo é instalar o Git no computador.

O processo é bastante simples e pode ser realizado através do instalador disponível no site oficial. Depois da instalação, o Git poderá ser utilizado diretamente pelo terminal.

Criando um repositório

Imagine que você acabou de criar um novo projeto em Python.

Para que o Git comece a acompanhar todas as alterações desse projeto, utilizamos o comando: `git init`

Esse comando cria um repositório Git dentro da pasta atual.

Em outras palavras, estamos dizendo:

"A partir de agora, quero que este projeto tenha controle de versão."

Verificando o status do projeto

Depois de criar um repositório, podemos verificar quais arquivos foram modificados utilizando: `git status`

Esse é um dos comandos mais utilizados pelos desenvolvedores.

Ele mostra quais arquivos foram criados, alterados ou ainda não foram adicionados ao controle de versão.

Sempre que tiver dúvidas sobre a situação do projeto, o `git status` provavelmente será seu best friend.

Adicionando arquivos

Depois de realizar alterações, precisamos informar ao Git quais arquivos deverão fazer parte da próxima versão do projeto. Para isso utilizamos: `git add .`

O ponto (.) indica que todos os arquivos modificados deverão ser adicionados.

Também é possível adicionar apenas um arquivo específico: `git add app.py`

Criando um commit

Agora chegamos a um dos conceitos mais importantes do Git. O **commit** funciona como uma fotografia do projeto naquele momento.

Sempre que concluimos uma funcionalidade ou realizamos uma alteração importante, criamos um novo commit.

Veja o comando: `git commit -m "Cadastro de clientes finalizado"`

Observe que informamos uma mensagem descrevendo a alteração realizada.

Essa mensagem será registrada no histórico do projeto.

No futuro, será muito mais fácil entender quando determinada funcionalidade foi adicionada.

Quando você precisar pensar em uma mensagem, você vai demorar mais tempo pensando no que escrever, do que escrevendo o código.

E o GitHub?

Git e GitHub não são a mesma coisa.

O Git é a ferramenta responsável por controlar as versões do projeto.

Já o GitHub é uma plataforma onde podemos armazenar nossos repositórios na internet.

Pense que o Git é como um editor de texto instalado no seu computador e o GitHub seria como um serviço de armazenamento em nuvem onde você pode guardar esse trabalho e compartilhá-lo com outras pessoas.

Embora sejam diferentes, eles costumam trabalhar juntos.

É muito comum utilizar o Git para controlar o projeto e o GitHub para armazená-lo e compartilhá-lo com equipes ou recrutadores.

Um exemplo do dia a dia

Agora, vamos ao momento desabafo.

Imagine que você está escrevendo o seu Trabalho de Conclusão de Curso. A cada semana você salva uma nova versão do documento.

TCC.docx

TCC_FINAL.docx

TCC_FINAL_AGORA_VAI.docx

TCC_FINAL_DEFINITIVO.docx

TCC_FINAL_DEFINITIVO_2.docx

Confessa...

Você já fez isso. Inclusive, utilizou os nomes “final” e “agora vai”.

O Git existe justamente para acabar com esse tipo de situação.

Em vez de criar dezenas de arquivos diferentes, ele registra todas as versões automaticamente dentro do mesmo projeto.

E eu confesso que preciso seriamente focar nisso, porque a mania do “agora_vai” NUNCA me abandona. Por sorte, muita sorte e amor no coração, não optei por trabalhar com desenvolvimento (apesar de ter que trabalhar com isso às vezes, não é 24h). Eu ia sofrer tanto com as minhas manias. Mas, preciso confessar que é muito mais organizado fazer do jeito certo, não acha?

HORA DE PRATICAR!

Tente antes de ver a resposta. Isso vai te ajudar mais do que você imagina!

1. O que é um ambiente virtual (venv) e qual é sua principal finalidade?

Resposta:

Um ambiente virtual é um espaço isolado criado para um projeto Python. Sua principal finalidade é manter as bibliotecas e dependências separadas dos demais projetos, evitando conflitos entre versões e tornando o desenvolvimento mais organizado.

2. Qual é a função do pip?

Resposta:

O pip é o gerenciador de pacotes oficial do Python. Ele permite instalar, atualizar, remover e gerenciar bibliotecas utilizadas pelos projetos.

3. Qual comando cria um ambiente virtual chamado venv?

Resposta:

```
python -m venv venv
```

4. Qual comando instala a biblioteca requests utilizando o pip?

Resposta:

```
pip install requests
```

5. Para que serve o arquivo requirements.txt?

Resposta:

Ele registra todas as bibliotecas utilizadas por um projeto, permitindo que outro desenvolvedor instale exatamente as mesmas dependências utilizando um único comando.

6. O que significa escrever um código utilizando boas práticas?

Resposta:

Significa desenvolver um código organizado, legível, bem estruturado, fácil de entender, manter e evoluir ao longo do tempo.

7. Qual é a principal função do Git?

Resposta:

Controlar as versões de um projeto, registrando todas as alterações realizadas e permitindo recuperar versões anteriores quando necessário.

8. Qual é a diferença entre Git e GitHub?

Resposta:

O Git é a ferramenta responsável pelo controle de versão do código. O GitHub é uma plataforma utilizada para armazenar e compartilhar repositórios Git na internet.

9. O que representa um *commit* no Git?

Resposta:

Um *commit* representa um registro das alterações realizadas em determinado momento do projeto, funcionando como uma "fotografia" daquela versão do código.

10. Por que ambientes virtuais, pip e Git são considerados ferramentas importantes no mercado de trabalho?

Resposta:

Porque ajudam a organizar projetos, controlar bibliotecas, evitar conflitos entre dependências, facilitar o trabalho em equipe e acompanhar toda a evolução do desenvolvimento de uma aplicação.

Desafio 1 — Preparando um Projeto Profissional

Imagine que você iniciará o desenvolvimento de um sistema em Python.

Descreva, em ordem, quais seriam os primeiros passos para preparar corretamente o ambiente de desenvolvimento.

Dica: lembre-se do ambiente virtual, instalação de bibliotecas e Git.

Desafio 2 — Organizando um Projeto

Você recebeu um projeto que possui todo o código em um único arquivo com mais de 2.000 linhas.

Explique quais boas práticas poderiam ser aplicadas para tornar esse projeto mais organizado, legível e fácil de manter.

Desafio 3 — Controle Financeiro Profissional

Imagine que você desenvolveu o Sistema de Controle Financeiro apresentado no projeto final desta trilha.

Agora pense como um desenvolvedor profissional.

Explique como você utilizaria os seguintes recursos nesse projeto:

- Ambiente virtual (venv);
- pip;
- Arquivo requirements.txt;
- Git.

Descreva qual seria a função de cada um deles durante o desenvolvimento da aplicação.

.

.

.

.

Possíveis Respostas dos Desafios

Desafio 1 — Preparando um Projeto Profissional

Possível resposta:

Ao iniciar um novo projeto, o primeiro passo é criar uma pasta para armazenar todos os arquivos da aplicação. Em seguida, é recomendado criar um ambiente virtual utilizando o comando `python -m venv venv`. Depois disso, o ambiente deve ser ativado para que todas as bibliotecas sejam instaladas apenas naquele projeto.

Com o ambiente ativo, podem ser instaladas as bibliotecas necessárias utilizando o pip. Também é uma boa prática iniciar um repositório Git com o comando `git init`, permitindo que todas as alterações realizadas no projeto sejam registradas desde o início.

Essa sequência ajuda a manter o projeto organizado, evita conflitos entre bibliotecas e facilita o trabalho em equipe.

Desafio 2 — Organizando um Projeto

Possível resposta:

Um projeto com mais de duas mil linhas em um único arquivo tende a ser difícil de entender e manter. Uma boa solução seria dividir o código em vários módulos, separando cada parte da aplicação por responsabilidade.

Também seria interessante criar funções menores e mais específicas, utilizar nomes claros para variáveis e funções, remover trechos de código repetidos e adicionar comentários apenas quando realmente forem necessários.

Outra prática importante seria organizar os arquivos em pastas, mantendo uma estrutura lógica para facilitar futuras alterações. Além disso, seguir um padrão de escrita, como as recomendações da PEP 8, torna o código mais legível e profissional.

Desafio 3 — Controle Financeiro Profissional

Possível resposta:

Se eu fosse desenvolver o Sistema de Controle Financeiro apresentado nesta apostila, começaria criando um ambiente virtual (venv) para isolar o projeto e evitar conflitos com outras aplicações instaladas no computador.

Sempre que fosse necessário utilizar uma biblioteca externa, faria a instalação utilizando o pip, garantindo que todas as dependências ficassem vinculadas ao ambiente virtual.

Depois de instalar as bibliotecas, geraria um arquivo requirements.txt, registrando todas as dependências utilizadas no projeto. Assim, qualquer outra pessoa poderia preparar o ambiente executando apenas um comando.

Também utilizaria o Git desde o início do desenvolvimento para registrar as alterações realizadas no código. A cada funcionalidade concluída, faria um novo *commit*, criando um histórico organizado da evolução do projeto e facilitando futuras modificações ou correções.

Essas práticas tornam o desenvolvimento mais organizado, seguro e muito mais próximo da realidade encontrada no mercado de trabalho.

Antes de me despedir, quero reforçar que os desafios apresentados nesta apostila não possuem apenas uma resposta correta. Em programação, é comum existirem diferentes maneiras de resolver o mesmo problema. O mais importante é compreender os conceitos estudados ao longo da trilha e justificar as decisões tomadas durante o desenvolvimento. Então, vamos seguir em frente!

AGRADECIMENTO

Se você chegou até aqui, quero te dizer: obrigada.

De verdade.

Finalizar um material assim não é só sobre aprender conteúdo. É sobre dedicação, tempo e vontade de evoluir. E só de você estar aqui, já mostra muito sobre você.

Essa apostila foi criada com um propósito muito claro: mostrar que conhecimento não precisa ser complicado. Que assuntos que parecem difíceis podem, sim, ser entendidos quando explicados da forma certa.

E mais do que isso: que conhecimento deve ser para todos.

Por isso, este material também foi pensado para ser acessível. Ele conta com versão em Braille, porque inclusão não é um diferencial — é o mínimo.

Todo mundo merece aprender. Todo mundo merece ter acesso.

Se, de alguma forma, esse conteúdo te ajudou a entender melhor, a se organizar ou até a enxergar as coisas de um jeito diferente, então ele já cumpriu o seu papel.

Obrigada por fazer parte disso.

E lembre-se: você não precisa complicar para fazer dar certo.

CONTATO

Se você quiser continuar aprendendo, trocar ideias ou acompanhar mais conteúdos sobre projetos, tecnologia e organização no dia a dia, você pode me encontrar por aqui:

Instagram: @CAROL_G_INTECH

LinkedIn: Caroline Gonçalves

Site: <https://cpgconsulting.com.br/>

E-mail: cpgtechconsulting@gmail.com

Fique à vontade para se conectar ou mandar uma mensagem. Vou adorar continuar essa troca com você.